

Glossary & cheatsheets

Appendices · [_shared/glossary/README.md](#) · [_shared/cheatsheets/README.md](#)

Glossary

One line per term — enough to jog your memory, not enough to replace the module that teaches it properly. Terms are grouped by the module that owns them; [→](#) points to that module's notes.

Machine learning (01 [\[01-ml-foundations\]](#))

- **Bias–variance tradeoff** — underfitting (bias) vs. overfitting (variance); the central tension in model capacity. [→ 01/notes/02](#)
- **Cross-validation** — splitting data into folds to get an honest, low-variance estimate of test performance. [→ 01/notes/02](#)
- **Data leakage** — information from outside the training set (often from the future, or the test set) contaminating training. [→ 01/notes/01](#)
- **Gradient descent** — iteratively stepping parameters against the loss gradient to minimize it. [→ 01/notes/02](#)
- **Regularization** — a penalty (L1/L2/dropout/early stopping) that trades train-fit for generalization. [→ 01/notes/02](#)

Deep learning (02 [\[02-deep-learning\]](#))

- **Backpropagation** — the chain rule applied systematically to compute gradients through a computation graph. [→ 02/notes/01](#)
- **Batch/Layer normalization** — normalizing activations to stabilize and speed up training. [→ 02/notes/01](#)
- **Residual connection** — $x + F(x)$; lets gradients skip layers, enabling very deep networks. [→ 02/notes/01](#)
- **Vanishing/exploding gradient** — gradients shrinking or growing exponentially across depth. [→ 02/notes/02](#)

NLP & transformers (03 [\[03-nlp\]](#))

- **Attention** — a weighted, content-based lookup: each token decides what to "look at" in the sequence. [→ 03/notes/02](#)
- **BPE (Byte-Pair Encoding)** — a subword tokenization algorithm that merges frequent character pairs. [→ 03/notes/01](#)
- **Embedding** — a dense vector representation of a discrete token, learned so that similar meanings land nearby. [→ 03/notes/01](#)
- **KV cache** — cached key/value tensors from prior tokens, reused during autoregressive decoding. [→ 03/notes/02](#)

- **RoPE (Rotary Position Embedding)** — encodes relative position by rotating query/key vectors. → [03/notes/02](#)

LLMs (04 [04-llm])

- **Chinchilla scaling** — the compute-optimal ratio of ~20 training tokens per parameter. → [04/notes/01](#)
- **DPO (Direct Preference Optimization)** — aligning a model to preferences without a separate reward model or RL loop. → [04/notes/01](#)
- **Hallucination** — fluent, confident output that is factually wrong or unsupported. → [04/notes/02](#)
- **RLHF** — Reinforcement Learning from Human Feedback; the classic alignment pipeline (reward model + PPO). → [04/notes/01](#)
- **Temperature / top-p** — sampling controls trading determinism for diversity at generation time. → [04/notes/02](#)

Agents & RAG (06 [06-ai-agents], 08 [08-rag])

- **ReAct** — interleaving reasoning ("thought") with tool calls ("action") in an agent loop. → [06/notes/01](#)
- **RAG (Retrieval-Augmented Generation)** — retrieving relevant context and inserting it into the prompt before generation. → [08/notes/01](#)
- **Reranking** — a second, more expensive scoring pass over an initial retrieval shortlist. → [08/notes/02](#)
- **Vector search / HNSW** — approximate nearest-neighbor search over embeddings for semantic retrieval. → [08/notes/01](#)

Responsible AI (12 [12-ai-governance]–15 [15-ai-evals])

- **Lethal trifecta** — an agent with untrusted input access + sensitive data access + external communication, all at once. → [13/notes/01](#)
- **LLM-as-judge** — using a model to score another model's output against a rubric. → [15/notes/01](#)
- **Model card** — a structured disclosure of a model's intended use, training data, evaluations, and limitations. → [12/notes/01](#)
- **Prompt injection** — malicious instructions smuggled into content a model processes as data. → [13/notes/01](#)
- **Risk tiering** — classifying an AI use case by potential harm to set proportional controls (core to the EU AI Act). → [12/notes/01](#), [14/notes/01](#)

Tech stack (16 [16-ai-tech-stack])

- **LCEL** — LangChain Expression Language; composing Runnables with `|`. → [16-ai-tech-stack/tracks/langchain](#)
- **MCP (Model Context Protocol)** — an open protocol connecting a model host to external tools/data/prompts. → [09-mcp](#)
- **State graph** — LangGraph's model of an agent as nodes + edges + persisted state, not a linear chain. → [16-ai-tech-stack/tracks/langgraph](#)

- **Trace / span** — a recorded tree of an application's execution, the unit LLM observability tools operate on.
→ [16-ai-tech-stack/tracks/langsmith](#)

Cheatsheets

Quick-reference tables pulled from across the track. Each links back to the module note that derives it — use these for recall, not for first learning.

Loss functions (01 [\[01-ml-foundations/notes/01-core-concepts.md\]](#))

Task	Loss	Notes
Regression	MSE / MAE / Huber	MSE penalizes outliers most; Huber compromises
Binary classification	Binary cross-entropy	pair with sigmoid
Multi-class	Categorical cross-entropy	pair with softmax; the LLM pretraining loss
Ranking / retrieval	Contrastive / InfoNCE	powers embedding models — see Module 08

Sampling parameters (04 [\[04-llm/notes/02-inference-and-adaptation.md\]](#))

Parameter	What it does	Typical range
Temperature	scales logits before softmax; higher = more random	0.0–1.2
top-p (nucleus)	samples from the smallest set of tokens covering p probability mass	0.9–0.95
top-k	restricts sampling to the k most likely tokens	20–50
Repetition penalty	discourages repeating already-generated tokens	1.0–1.3

Attention shapes (03 [\[03-nlp/notes/02-transformers.md\]](#))

Symbol	Meaning
<code>d_model</code>	the residual stream width (e.g. 4096)
<code>n_heads</code>	number of attention heads
<code>d_head</code>	<code>d_model / n_heads</code> , the per-head dimension
<code>n_ctx</code>	context length in tokens
KV cache size	<code>2 × n_layers × n_kv_heads × d_head × n_ctx × batch × bytes_per_elem</code>

Model adaptation decision order (04 [\[04-llm/notes/02-inference-and-adaptation.md\]](#))

Prompting → Few-shot → RAG (08) → LoRA fine-tune → Full fine-tune → Continued pretraining — try each in order; only move to the next when the current one demonstrably fails on your eval set (15).

RAG retrieval metrics (08 [08-rag])

Metric	Measures
Recall@k	fraction of relevant docs found in the top k
MRR	how high the FIRST relevant doc ranks, averaged
NDCG	ranking quality weighted by graded relevance
Faithfulness	does the generated answer only use retrieved content

OWASP-style LLM risk categories (13 [13-ai-security])

Prompt injection (direct/indirect) · insecure output handling · training data poisoning · model denial of service · supply chain vulnerabilities · sensitive info disclosure · insecure plugin/tool design · excessive agency · overreliance · model theft. See [13-ai-security/notes/](#) for the verified current list and mitigations for each.

Statistical significance for evals (15 [15-ai-evals])

- A score difference on $n < 100$ examples is usually noise — compute a confidence interval (Wilson score interval for proportions) before trusting it.
- Use a **paired** test (McNemar's, paired bootstrap) when comparing two models/prompts on the *same* examples — it's far more powerful than comparing independent means.
- Binary rubric criteria produce better human/judge agreement (Cohen's kappa) than 1–10 Likert scales.